
Classification d'images par réseaux de neurones convolutifs (CNN)

Jeu de données CIFAR-10 : architectures VGG, optimiseurs, learning rate et régularisation

Mehdi Redha BELLAHSENE

Méthodologie CRISP-DM . Python / TensorFlow / Keras . Juin 2026

Application : <https://machine-learning.bellahsene.org>

Code source : <https://github.com/mehdibellahsene/projet-ml-cifar10>

Résumé

Ce travail étudie la classification d'images CIFAR-10 par CNN. Nous comparons une architecture de base (LeNet-5) à des VGG (1 à 3 blocs), évaluons l'effet de l'optimiseur (SGD, Adam) et du learning rate, puis l'apport de la régularisation (Dropout, Batch Normalization), selon CRISP-DM. La meilleure configuration imposée (VGG3+D+BN, Adam, LR=0.001) atteint 82.8 %. En allant plus loin, le transfer learning (EfficientNetB5_TL) porte la précision à 97.9 %, le meilleur résultat du projet.

1. Introduction et méthodologie

L'objectif de ce projet est d'implémenter des réseaux de neurones convolutifs (CNN) pour la classification d'images du jeu CIFAR-10 : étude des architectures de type VGG, comparaison des optimiseurs SGD et Adam sur trois learning rates, et évaluation des techniques de régularisation (Dropout, Batch Normalization).

1.1 Méthodologie CRISP-DM (les six phases)

La démarche suit la méthodologie CRISP-DM (Cross-Industry Standard Process for Data Mining), un processus itératif en six phases ; voici comment chacune se concrétise dans ce projet :

- Compréhension du problème : classer des images couleur 32x32 en 10 catégories mutuellement exclusives (tâche de vision par ordinateur).
- Compréhension des données : exploration de CIFAR-10 -- dimensions, exemples visuels, distribution des classes (section 2).
- Préparation des données : normalisation des pixels dans $[0,1]$ et encodage one-hot des labels (section 2).
- Modélisation : construction de LeNet-5, VGG1/2/3 et des variantes régularisées ; choix des optimiseurs et des learning rates (section 3).
- Évaluation : tableau de performances, courbes d'apprentissage et analyse comparative des résultats (section 4).
- Déploiement : sélection du modèle retenu (section 5) et extensions pour la précision maximale (sections 6-7) ; l'application web ML Playground met le modèle à l'épreuve (section 8).

1.2 Environnement d'exécution et consommation des ressources

Les entraînements ont été réalisés en Python (TensorFlow / Keras) sur Google Colab, avec un GPU NVIDIA A100 (80 Go de mémoire GPU) dans un environnement à HAUTE MÉMOIRE (High-RAM, 167 Go de RAM système). Cette configuration High-RAM a été nécessaire pour entraîner le modèle de transfer learning EfficientNetB5 à haute résolution (456x456) sans saturation de la mémoire. Le transfer learning emploie en outre la précision mixte (float16) pour exploiter les Tensor Cores du GPU et accélérer l'entraînement.

Point sur l'exécution. La grille des 36 configurations (modèles légers) s'entraîne en quelques minutes ; à l'inverse, EfficientNetB5 -- le plus gros modèle -- sollicite fortement le GPU (environ 18 minutes par époque en fine-tuning à 456x456). La capture ci-dessous montre la consommation des ressources (RAM système, mémoire GPU, disque) pendant l'exécution sur l'environnement A100 High-RAM.

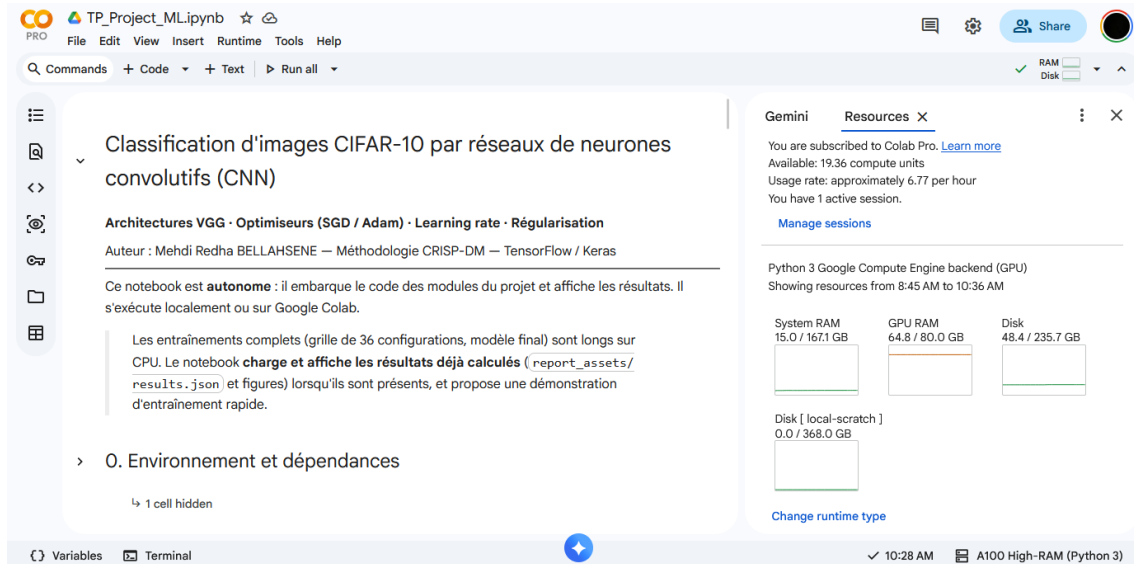


Figure. Consommation des ressources sur Colab (A100, environnement High-RAM).

2. Compréhension et préparation des données

2.1 Jeu de données (Tâche 1)

CIFAR-10 : images couleur 32x32 (3 canaux RVB), 10 classes mutuellement exclusives.

Caractéristique	Valeur
Images d'entraînement	50 000
Images de test	10 000
Dimensions image	32 x 32 x 3
Canaux	3
Classes	10

Table 1. Caractéristiques de CIFAR-10.

50 000 images d'entraînement et 10 000 de test. La faible résolution rend la tâche difficile (classes proches : chat/chien, automobile/camion).

2.2 Visualisation (Tâche 2)

Les labels sont des données CATÉGORIELLES (et non numériques ordonnées) : chaque entier de 0 à 9 désigne une catégorie (avion, automobile, ...), sans relation d'ordre entre elles. Visuellement, la très faible résolution (32x32) rend certaines images difficiles à classer, y compris pour un humain -- notamment des classes proches comme chat / chien ou automobile / camion.

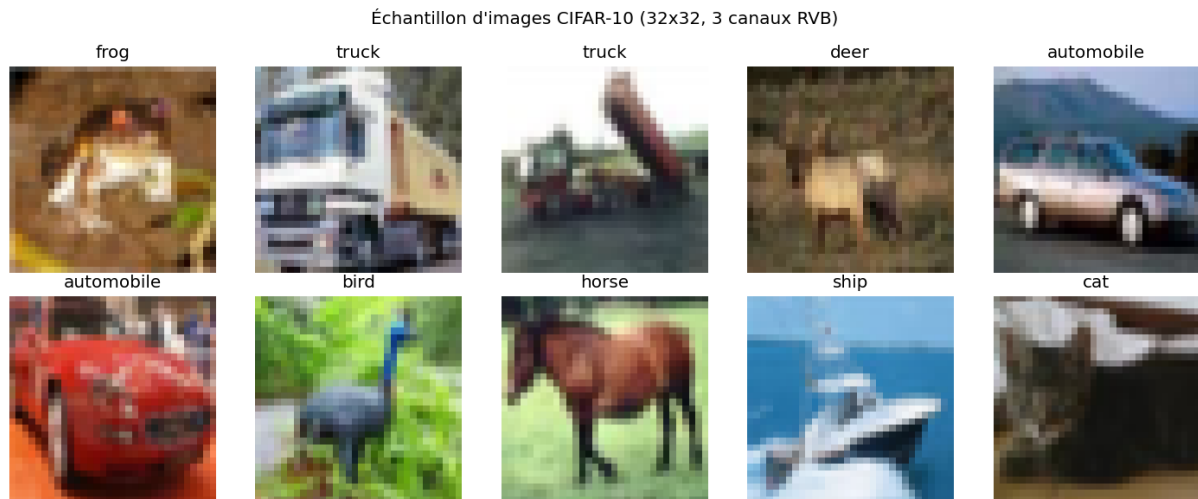


Figure 1. Échantillon d'images CIFAR-10 et leurs classes.

2.3 Normalisation (Tâche 3)

Les pixels (0-255) sont ramenés dans $[0,1]$ par division par 255. Cela place les entrées sur une échelle commune, stabilise et accélère la descente de gradient.

2.4 Encodage one-hot (Tâche 4)

Les labels deviennent des vecteurs one-hot (classe 3 $\rightarrow [0,0,0,1,0,0,0,0,0]$). Cela évite toute relation d'ordre entre classes et s'accorde avec softmax + entropie croisée.

2.5 Distribution des classes (Tâche 5)

CIFAR-10 est équilibré : 5 000 images par classe. Un déséquilibre biaiserait le modèle.

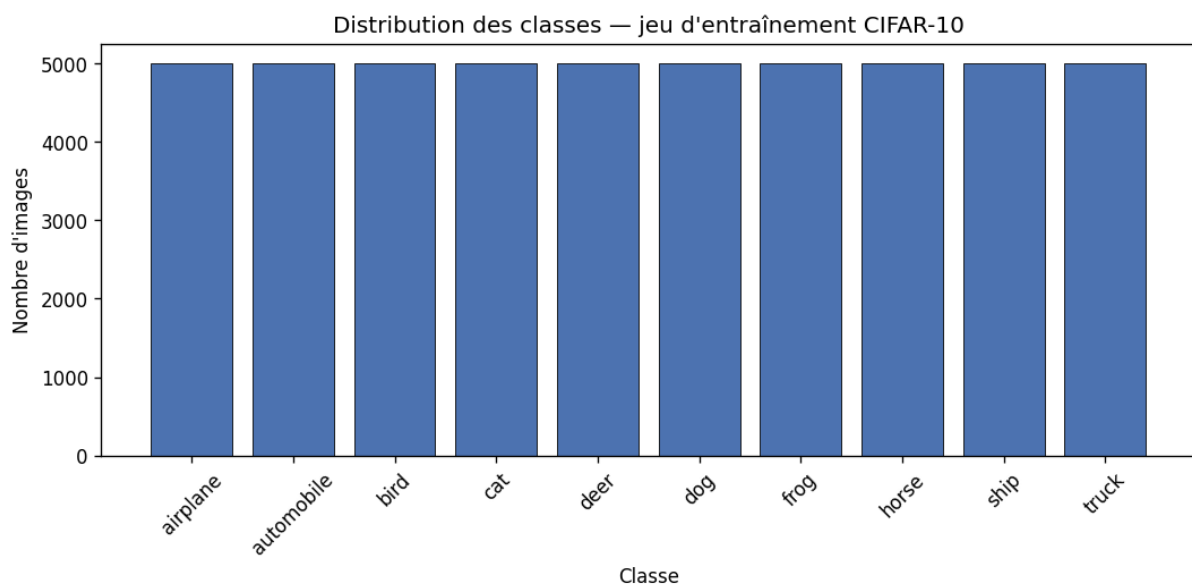


Figure 2. Distribution des classes (5 000 chacune).

3. Modélisation : architectures, optimiseurs, régularisation

3.1 Architectures

Les architectures de type VGG (Simonyan & Zisserman, 2014) empilent des blocs convolutifs identiques. Un bloc VGG = Conv $\rightarrow$ Conv $\rightarrow$ MaxPool (32 filtres 3x3, pooling 2x2) ; suit une couche cachée Dense(128) puis la sortie softmax(10). Six modèles :

- CNN1 : réseau de base (LeNet-5), référence.
- VGG1/2/3 : 1, 2 ou 3 blocs (profondeur croissante).
- VGG3+Drop : VGG3 + Dropout.
- VGG3+Drop+BatchNorm : VGG3 + Dropout + Batch Normalization.

Modèle	Blocs VGG	Paramètres
CNN1	-	136 886
VGG1	1	1 060 138
VGG2	2	292 202
VGG3	3	114 090
VGG3+D	3	114 090
VGG3+D+BN	3	115 370

Table 2. Nombre de paramètres par architecture.

3.2 Optimiseurs (SGD et Adam)

Un optimiseur est l'algorithme qui ajuste les poids du réseau afin de minimiser la fonction de perte. À chaque itération, il exploite les gradients calculés par rétropropagation pour mettre à jour chaque poids dans la direction qui réduit l'erreur. Son rôle est central : il détermine la vitesse, la stabilité et la qualité finale de l'apprentissage.

- SGD (descente de gradient stochastique) met à jour chaque poids d'un pas fixe : $w \leftarrow w - lr \times \text{gradient}$. Simple et robuste, mais sensible au choix du learning rate et plus lent ; le momentum (0,9) accumule les gradients passés pour accélérer et lisser la trajectoire.
- Adam (Adaptive Moment Estimation) combine le momentum (moyenne des gradients) et un learning rate adaptatif par paramètre (via la moyenne des carrés des gradients) : $w \leftarrow w - lr \times m / (\text{racine}(v) + \text{eps})$. Il converge plus vite et demande moins de réglage, d'où son statut de choix par défaut pour les CNN.

Quel optimiseur pour l'architecture VGG ? L'analyse (section 4.3) le montre : Adam est le plus performant à faible learning rate (0.001), mais il diverge à learning rate élevé, où SGD reste plus robuste. (Réfs : Kingma & Ba, 2015 ; Analytics Vidhya, 2021.)

3.3 Régularisation : Dropout et Batch Normalization (Bonus)

Dropout. Pendant l'entraînement, le Dropout désactive aléatoirement une fraction des neurones (20 à 50 % ici) à chaque étape : le réseau ne peut plus dépendre de neurones spécifiques et apprend des représentations redondantes et robustes, ce qui réduit le surapprentissage. Différence essentielle entre les deux phases : le Dropout n'agit qu'à l'ENTRAÎNEMENT ; en INFÉRENCE, tous les neurones sont conservés et leurs sorties mises à l'échelle de manière cohérente. (Réf : Srivastava et al., 2014.)

Batch Normalization. Elle normalise les activations de chaque couche au sein d'un mini-batch (moyenne nulle, variance unitaire), puis les remet à l'échelle via deux paramètres appris. Objectifs et bénéfices : réduire le décalage de distribution interne (covariate shift), ce qui stabilise et accélère l'entraînement, autorise des

learning rates plus élevés et exerce un léger effet régularisant. (Réf : Ioffe & Szegedy, 2015.)

3.4 Protocole

Entropie croisée catégorielle, batch 64, max 50 époques, early stopping. Trois learning rates (0.001, 0.01, 0.1) x deux optimiseurs ; LR fixe pendant chaque entraînement. Le test sert de validation.

4. Implémentation et évaluation des performances

4.1 Tableau des performances (précision de test)

Précision test (%)	LR = 0.001		LR = 0.01		LR = 0.1	
	<i>SGD</i>	<i>Adam</i>	<i>SGD</i>	<i>Adam</i>	<i>SGD</i>	<i>Adam</i>
CNN1	64.3	64.1	63.9	53.1	10.0	10.0
VGG1	66.6	68.6	68.3	45.0	44.2	10.0
VGG2	67.7	74.3	70.8	54.0	10.0	10.0
VGG3	71.7	75.5	73.1	10.0	10.0	10.0
VGG3+D	71.4	78.6	76.0	10.0	10.0	10.0
VGG3+D+BN	78.2	82.8	82.8	81.3	81.8	59.6

Table 3. Précision de test (%). CNN1 = LeNet-5 ; VGG3+D = VGG3 + Dropout ; VGG3+D+BN = VGG3 + Dropout + Batch Normalization.

Meilleure configuration : VGG3+D+BN / Adam / LR=0.001 -> 82.82 % (entraînement 89.03 %).

4.2 Perte de test

Perte test	LR = 0.001		LR = 0.01		LR = 0.1	
	<i>SGD</i>	<i>Adam</i>	<i>SGD</i>	<i>Adam</i>	<i>SGD</i>	<i>Adam</i>
CNN1	1.14	1.06	1.10	1.40	2.31	2.31
VGG1	1.25	0.91	0.96	1.55	1.77	2.31
VGG2	1.01	0.90	1.00	1.30	2.30	2.31
VGG3	0.93	0.79	0.82	2.30	2.31	2.31
VGG3+D	0.80	0.62	0.71	2.30	2.31	2.31
VGG3+D+BN	0.63	0.51	0.51	0.55	0.56	1.18

Table 4. Perte (entropie croisée). CNN1 = LeNet-5 ; VGG3+D = VGG3 + Dropout ; VGG3+D+BN = VGG3 + Dropout + Batch Normalization.

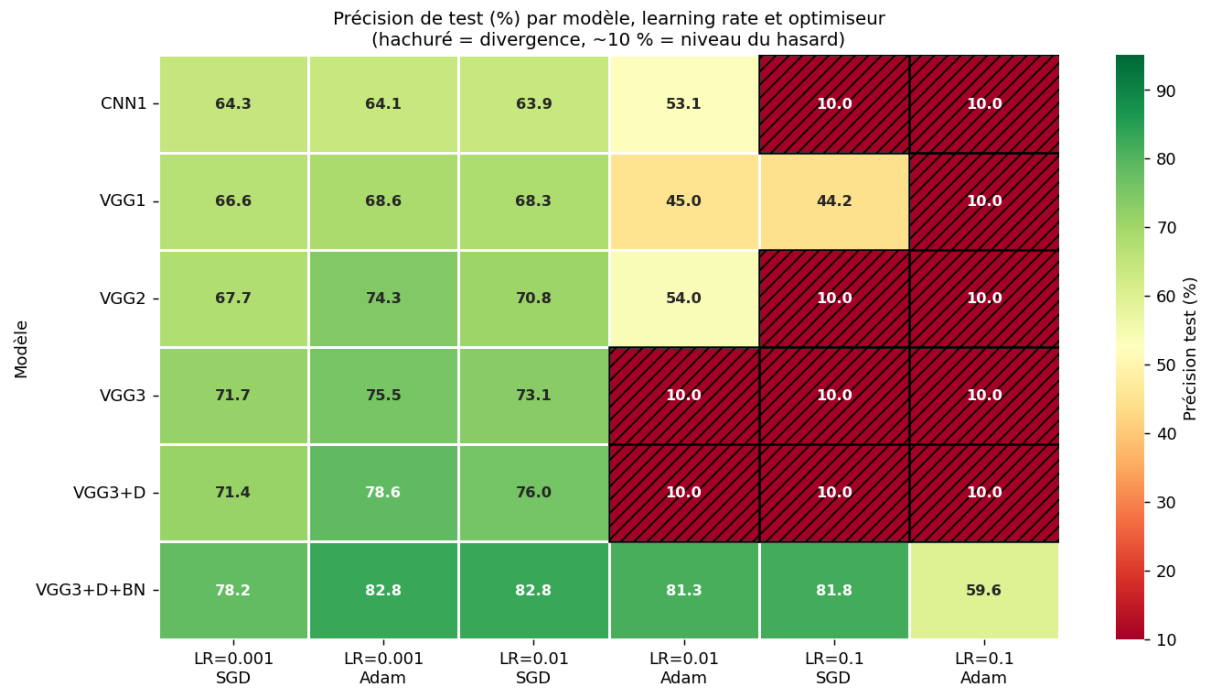


Figure 3. Précision de test (heatmap) : modèles x (learning rate, optimiseur).

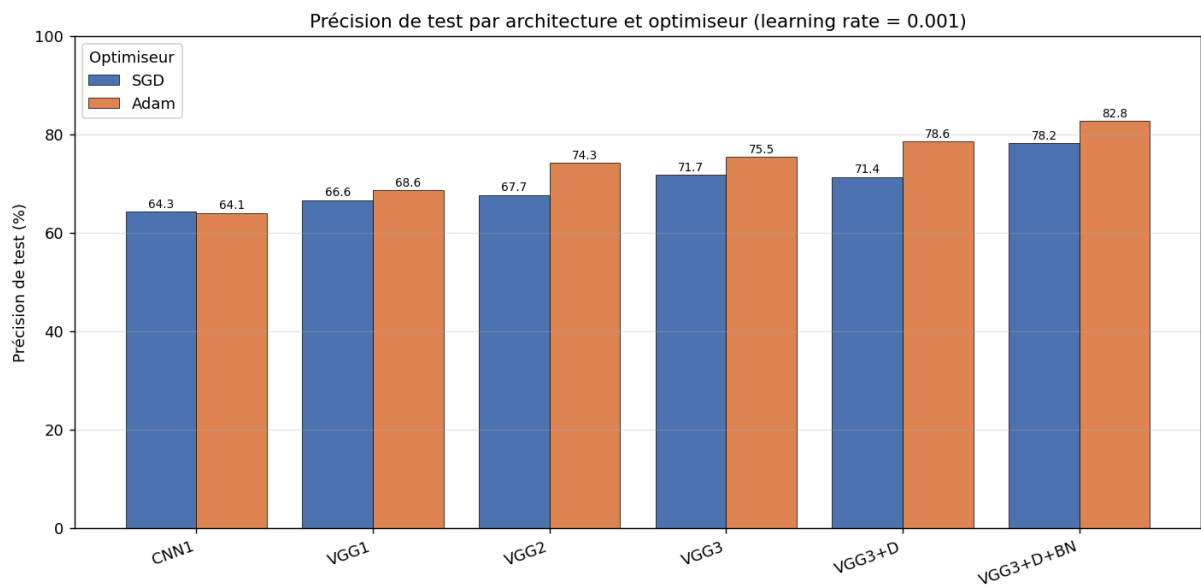


Figure 4. Précision de test par architecture et optimiseur (au meilleur learning rate).

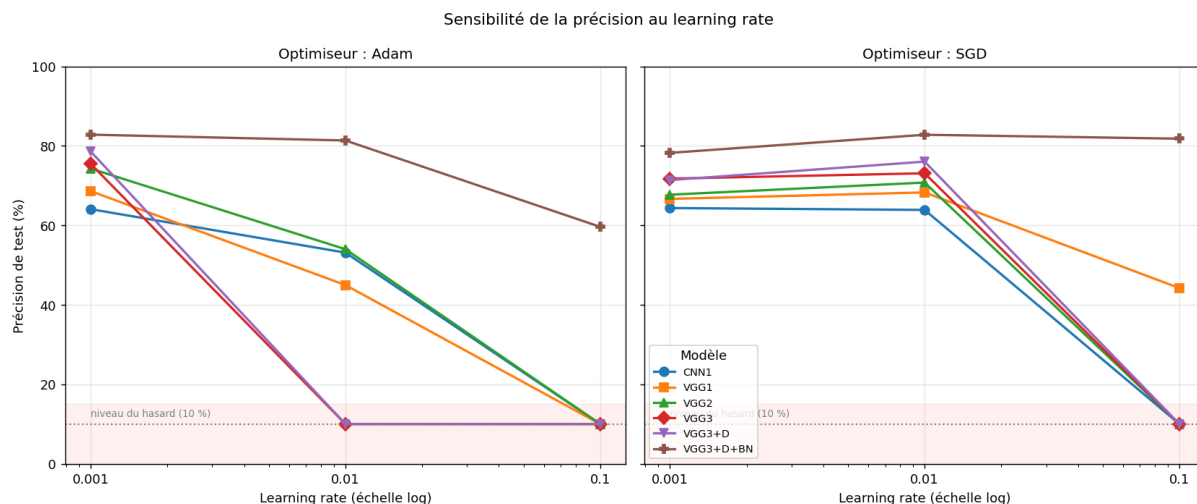


Figure 5. Sensibilité de la précision au learning rate, par optimiseur.

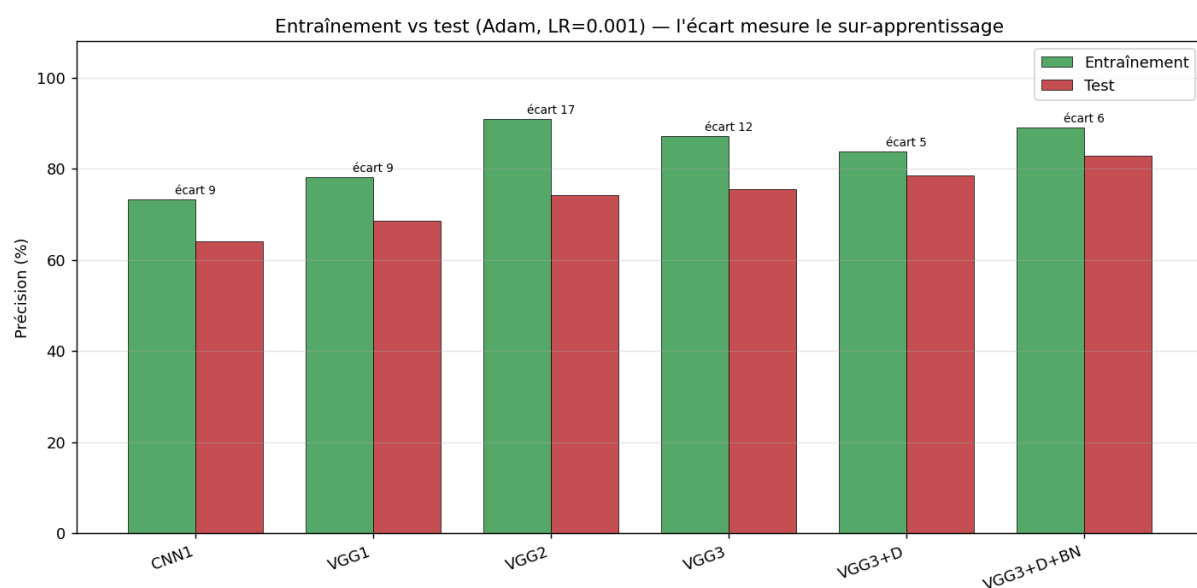


Figure 6. Entraînement vs test : l'écart mesure le sur-apprentissage.

4.3 Analyse des résultats

Effet de l'architecture. Les performances progressent de CNN1 (LeNet-5) vers les VGG avec la profondeur : CNN1 64.3 % ; VGG1 68.6 % ; VGG2 74.3 % ; VGG3 75.5 % ; VGG3+D 78.6 % ; VGG3+D+BN 82.8 %. Empiler des blocs convolutifs accroît la capacité d'extraction de caractéristiques, donc la précision, jusqu'à un palier.

Effet de l'optimiseur. Le meilleur optimiseur dépend du learning rate. Au learning rate adapté (LR=0.001), Adam surpasse SGD (ex. VGG3 à LR=0.001 : Adam 75.5 % contre SGD 71.7 %). Mais à 0.01 et 0.1, Adam devient instable et diverge, alors que SGD reste robuste : 7 divergence(s) pour Adam contre 4 pour SGD. Gagnant par learning rate : LR=0.001 : Adam (74 %) ; LR=0.01 : SGD (72 %) ; LR=0.1 : SGD (28 %). Adam est le meilleur à faible LR, SGD plus sûr à LR élevé.

Effet du learning rate. Précision moyenne par LR : LR=0.001 -> 72.0 % ; LR=0.01 -> 57.4 % ; LR=0.1 -> 23.0 % ; meilleur compromis LR=0.001. Un LR trop élevé fait diverger Adam (précision ~10 %, niveau du hasard) : son pas adaptatif sature la sortie softmax. SGD reste stable. La sensibilité au learning rate est une faiblesse d'Adam et une force de SGD.

Effet de la régularisation (VGG3, Adam, LR=0.001). En partant de VGG3 (75.5 % (écart train-test 11.7 pts)),

l'ajout du Dropout donne 78.6 % (écart train-test 5.2 pts), puis l'ajout de la Batch Normalization donne 82.8 % (écart train-test 6.2 pts). La régularisation réduit le surapprentissage ; la Batch Normalization stabilise en outre l'entraînement et autorise des learning rates plus élevés.

Batch Normalization et robustesse au learning rate. À $LR=0.1$, seul le modèle avec Batch Normalization évite la divergence (jusqu'à 81.8 %), tandis que les autres s'effondrent au niveau du hasard (au mieux 44 %). C'est la démonstration du bénéfice central de la BN : en normalisant les activations, elle empêche l'explosion des gradients et autorise des learning rates bien plus élevés.

4.4 Courbes d'apprentissage (learning rate = 0.001)

Perte et précision (entraînement + test) pour chaque modèle, sous SGD puis Adam.

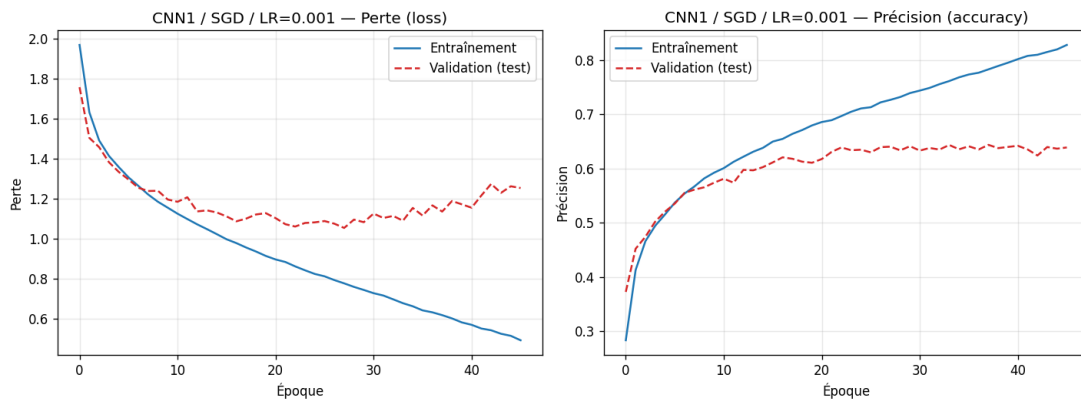


Figure 7. CNN1 / SGD / LR=0.001.

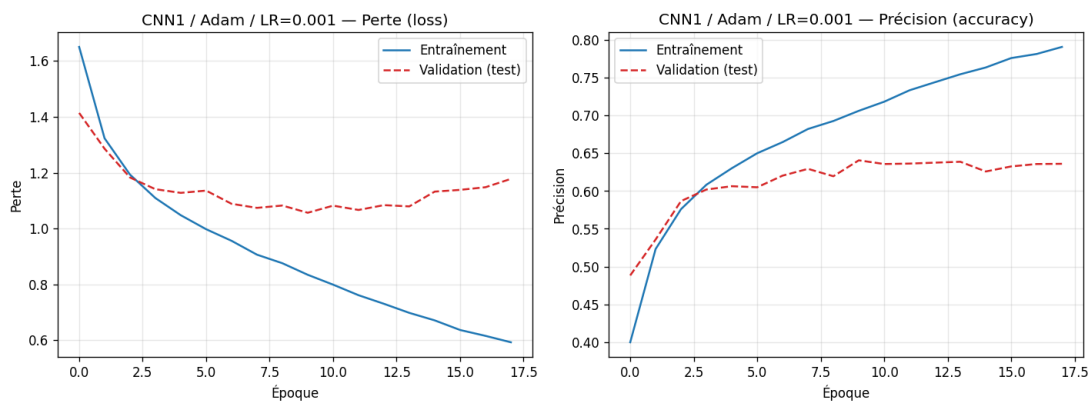


Figure 8. CNN1 / Adam / LR=0.001.

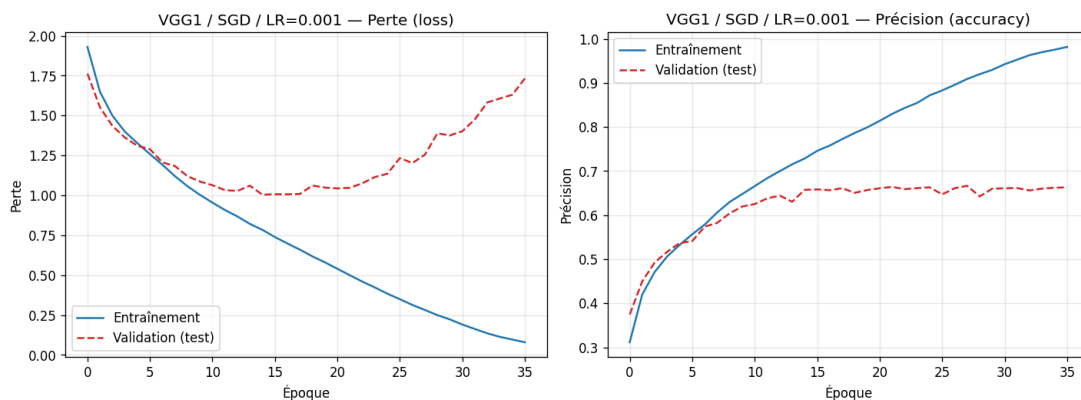


Figure 9. VGG1 / SGD / LR=0.001.

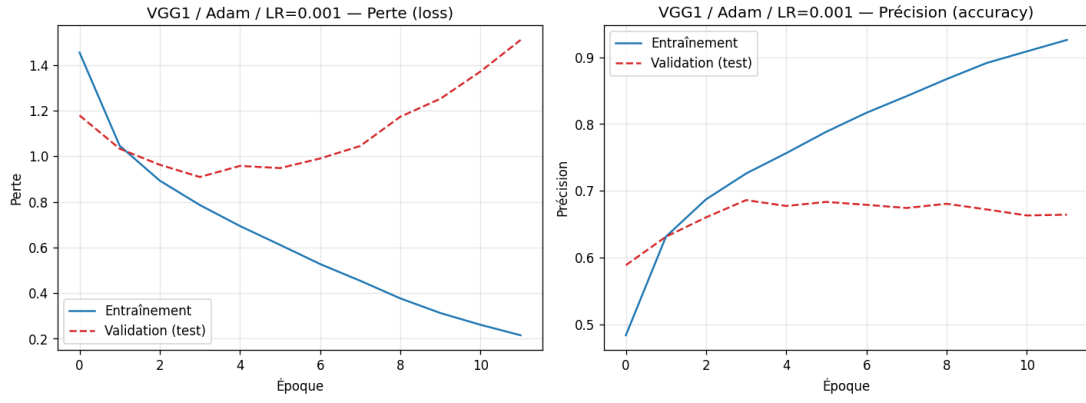


Figure 10. VGG1 / Adam / LR=0.001.

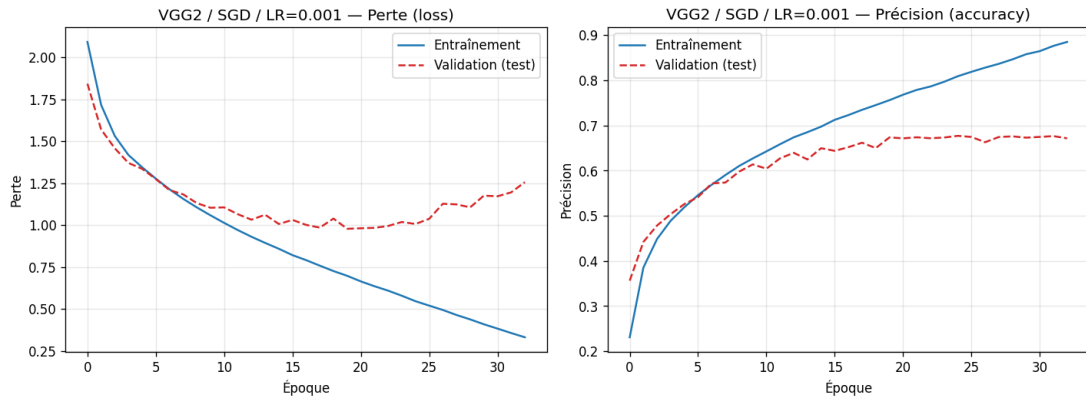


Figure 11. VGG2 / SGD / LR=0.001.

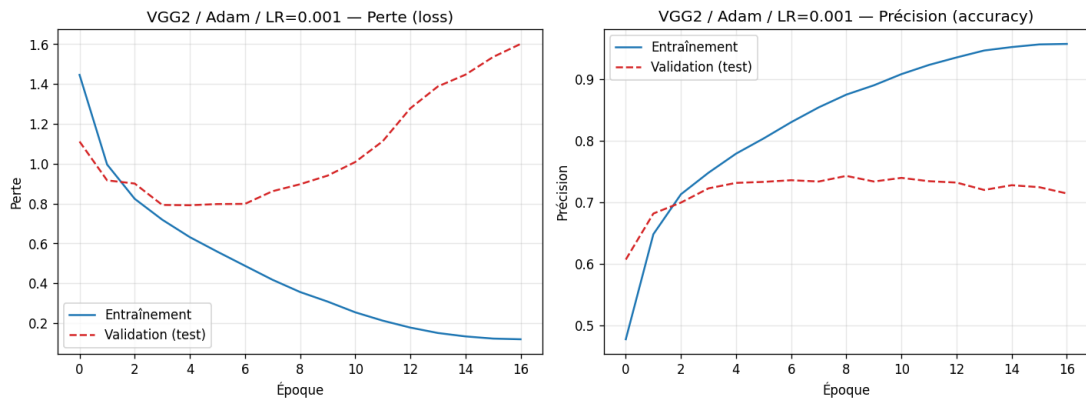


Figure 12. VGG2 / Adam / LR=0.001.

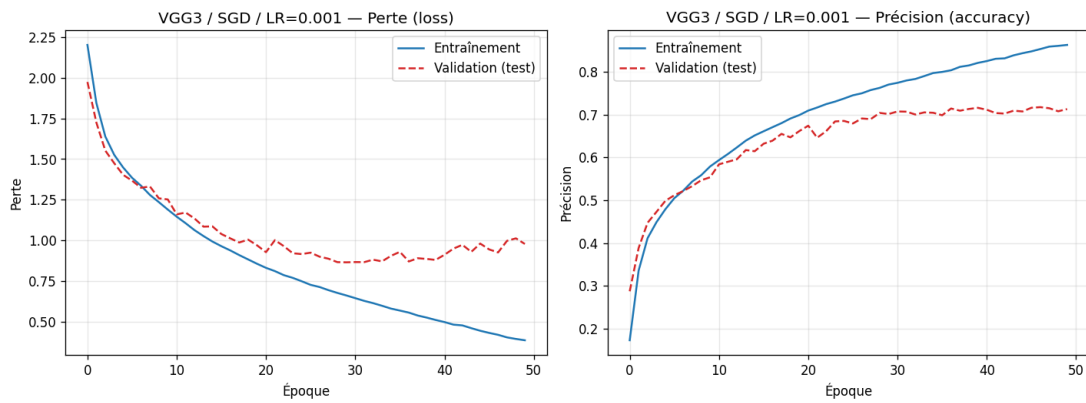


Figure 13. VGG3 / SGD / LR=0.001.

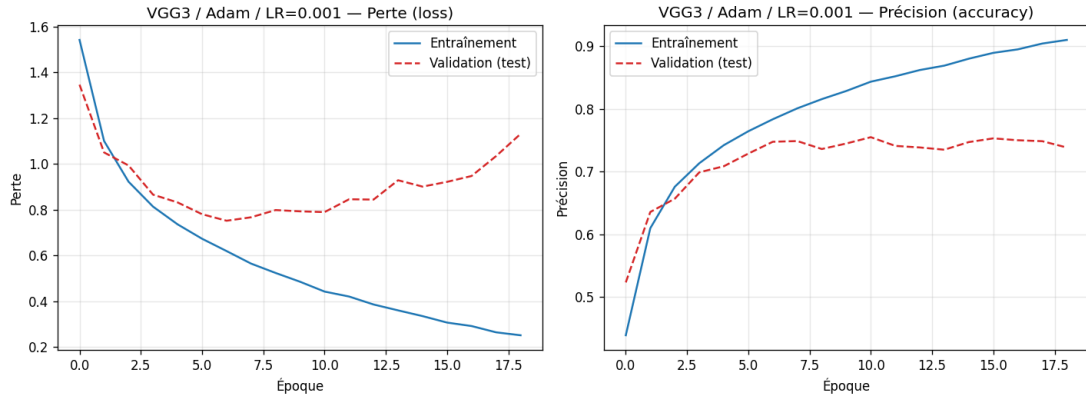


Figure 14. VGG3 / Adam / LR=0.001.

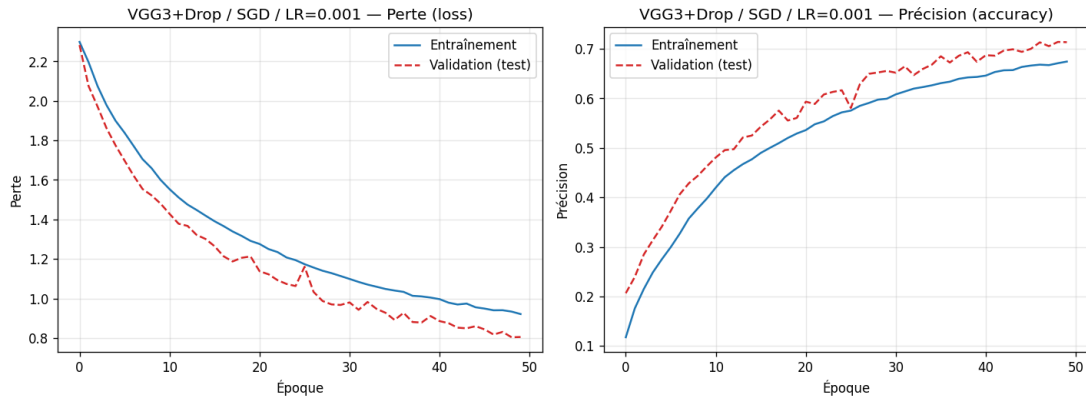


Figure 15. VGG3+D / SGD / LR=0.001.

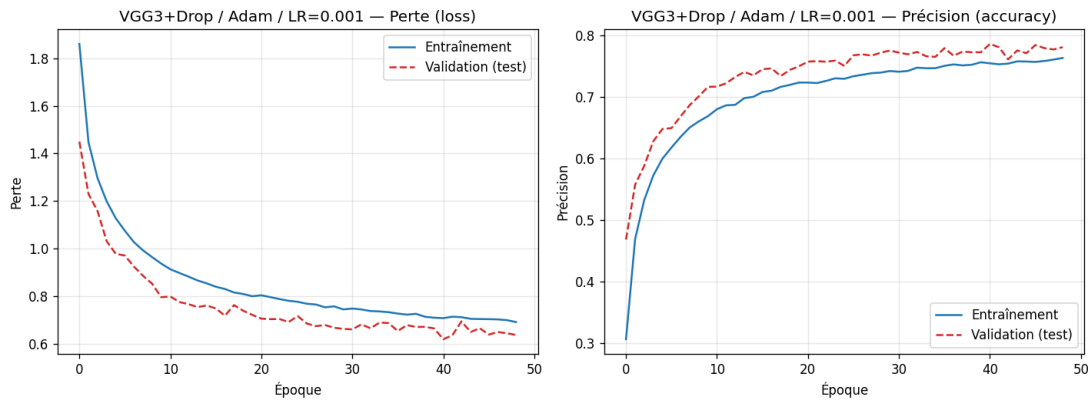


Figure 16. VGG3+D / Adam / LR=0.001.

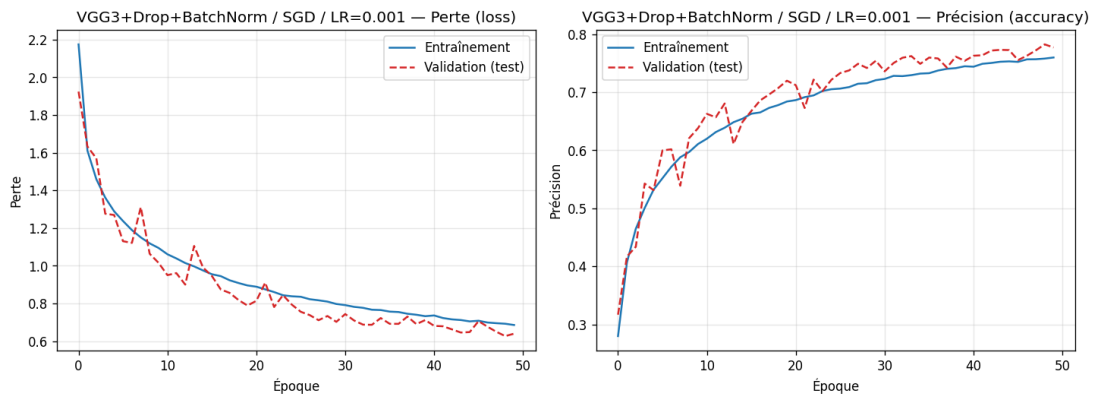


Figure 17. VGG3+D+BN / SGD / LR=0.001.

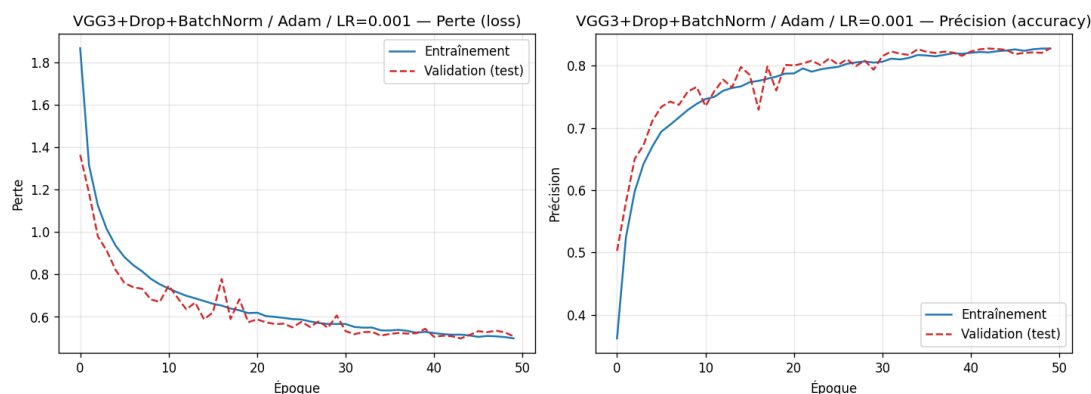


Figure 18. VGG3+D+BN / Adam / LR=0.001.

5. Modèle retenu

Conformément à l'énoncé, le modèle retenu est la meilleure configuration : VGG3+D+BN / Adam / LR=0.001, réentraîné plus longtemps (n'utilise que les architectures imposées).

Indicateur	Valeur
Architecture	VGG3+D+BN
Optimiseur / LR	Adam / 0.001
Précision de test	84.81 %
Perte de test	0.460
Paramètres	115 370
Époques	88 / 100

Table 5. Modèle retenu.

À noter : même réentraîné longtemps (jusqu'à 100 époques), ce modèle plafonne autour de 84-85 %. Ce n'est pas un défaut d'entraînement mais une limite de CAPACITÉ : l'architecture imposée est volontairement petite (blocs VGG à 32 filtres seulement, ~114 000 paramètres). La précision d'entraînement et la précision de test restent proches (autour de 85 %), ce qui indique une sous-capacité et non du surapprentissage : le réseau atteint sa limite de représentation. Au-delà, ce n'est plus le nombre d'époques qui compte mais la capacité du modèle -- d'où les extensions des sections 6 et 7 (architecture plus large : 89.6 % ; transfer learning : 97.9 %).

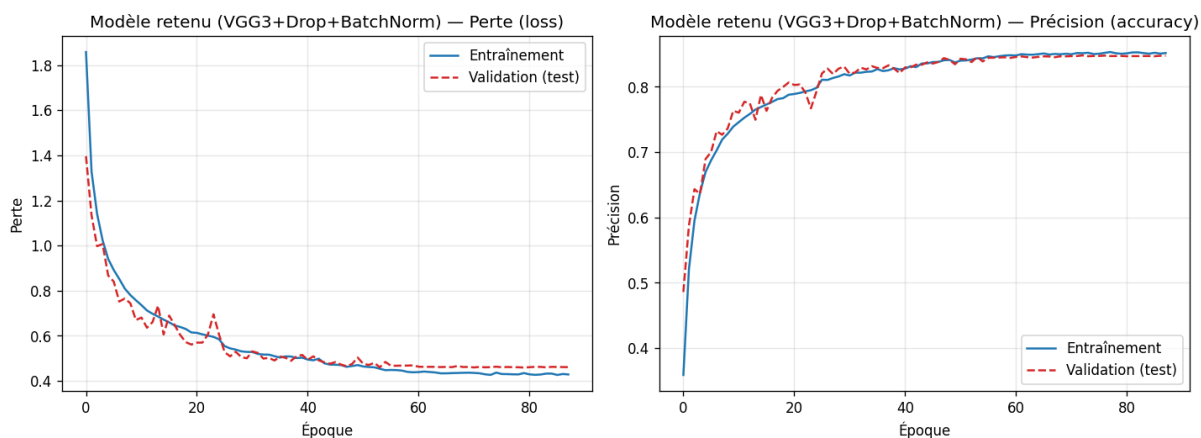


Figure. Courbes du modèle retenu.

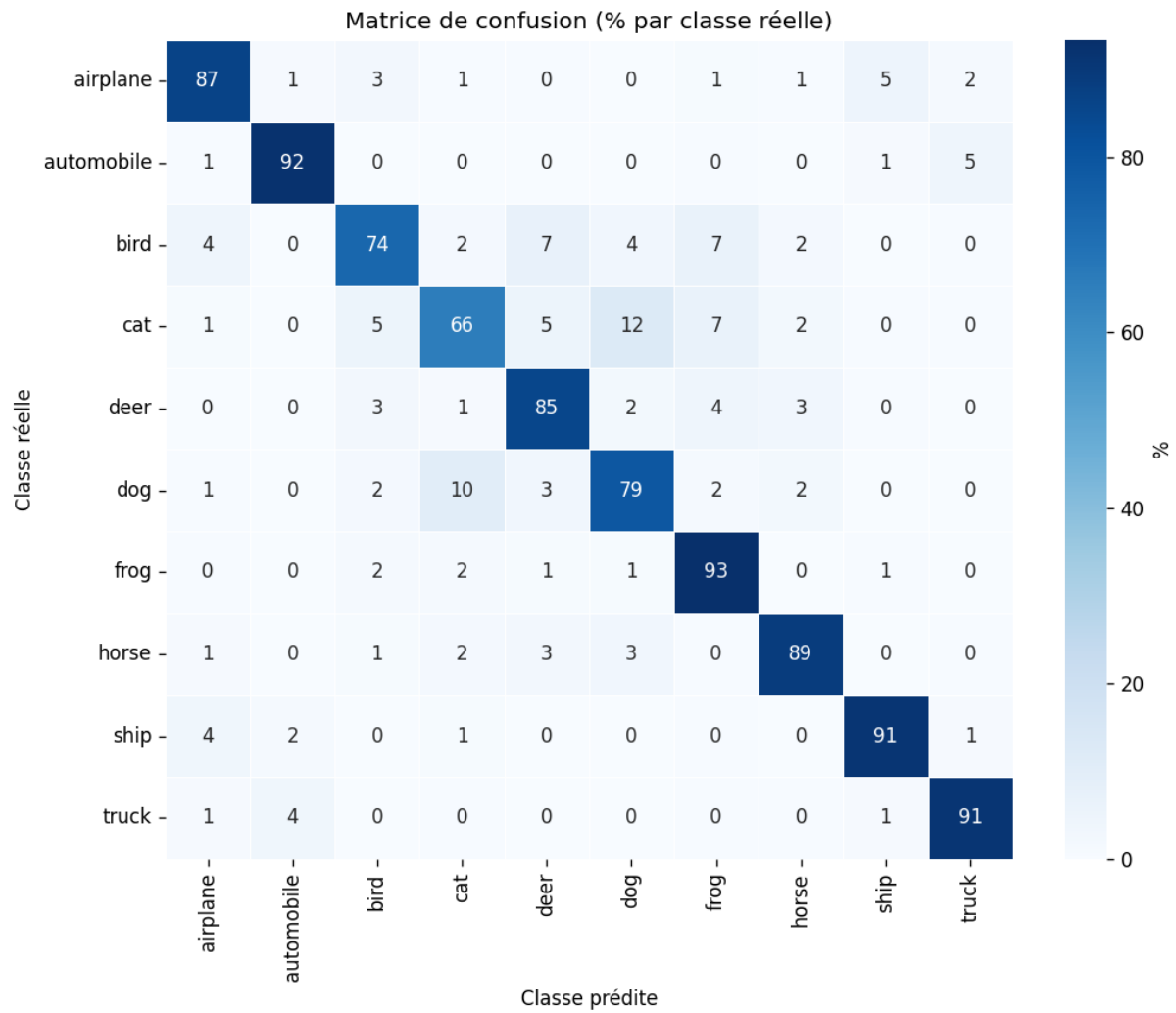


Figure. Matrice de confusion du modèle retenu (%).

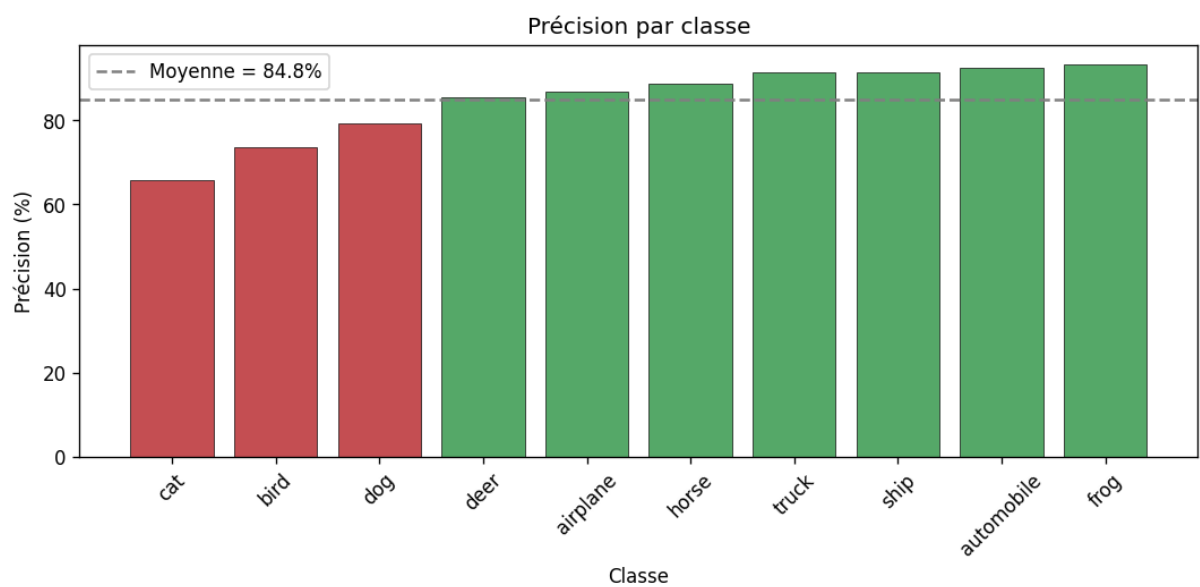


Figure. Précision par classe du modèle retenu.

Classe la mieux reconnue : grenouille (93.3 %) ; la plus difficile : chat (66.7 %). Confusions entre classes proches.

6. Bonus 1 : transfer learning (modèle plus léger)

Première extension (hors énoncé) par transfer learning : un réseau EfficientNetB0 (transfer learning) pré-entraîné sur ImageNet, dont les images CIFAR-10 sont agrandies puis le réseau affiné. Backbone plus léger (entraînement rapide), déjà très performant grâce au transfert de connaissances.

Indicateur	Valeur
Modèle	EfficientNetB0 (transfer learning)
Précision de test	96.64 %
Perte de test	0.099
Paramètres	4 062 381
Époques	25 / 25

Table 6. Modèle transfer learning (léger).

Ce modèle atteint 96.64 %, au-delà des architectures imposées.

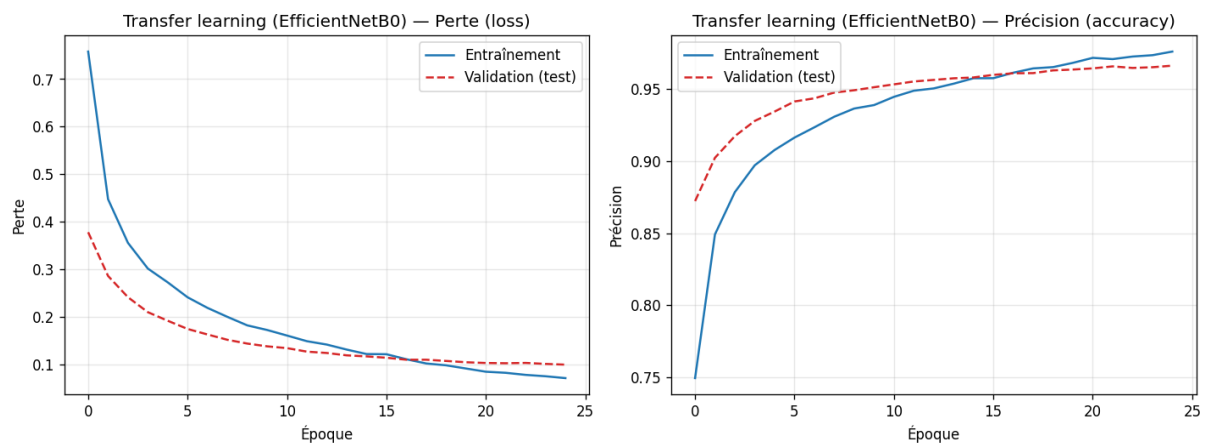


Figure. Courbes (Modèle transfer (léger)).

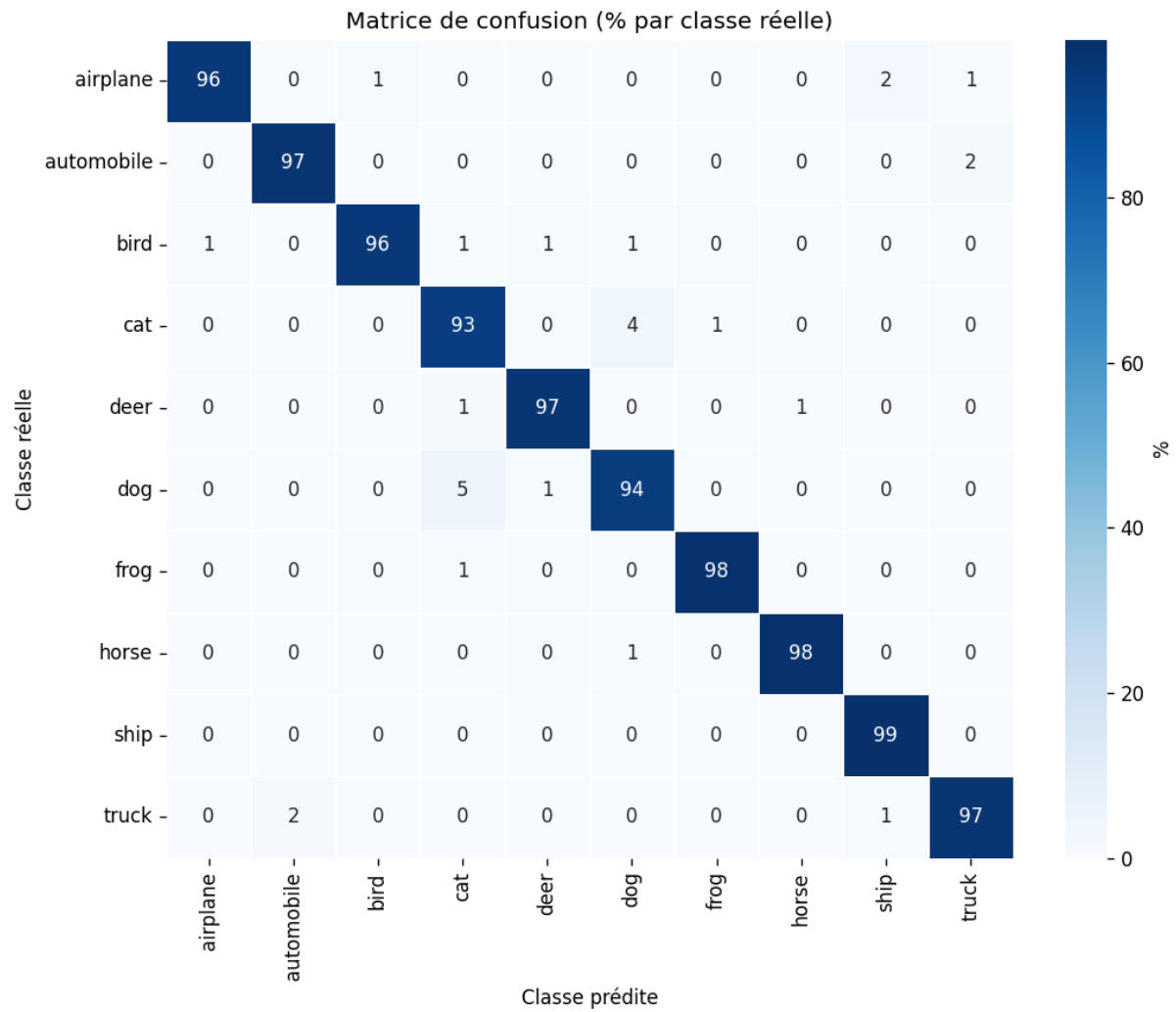


Figure. Matrice de confusion (Modèle transfer (léger), %).

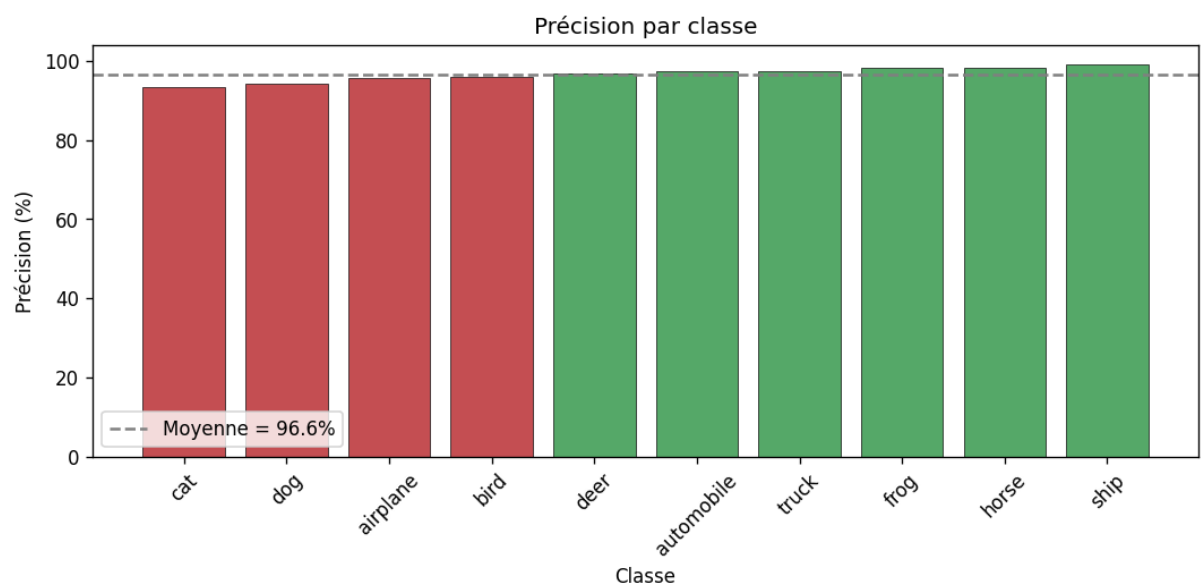


Figure. Précision par classe (Modèle transfer (léger)).

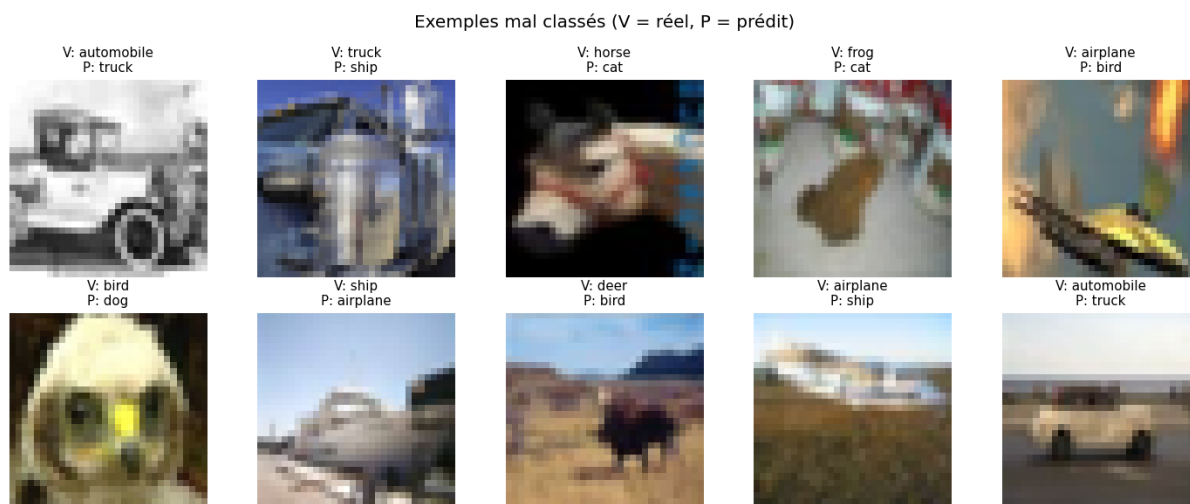


Figure. Exemples mal classés (V = réel, P = prédit).

7. Bonus 2 : transfer learning (précision maximale)

Seconde extension visant la précision maximale : le transfer learning. Un réseau EfficientNetB5_TL pré-entraîné sur ImageNet est réutilisé ; les images CIFAR-10 (32x32) sont agrandies à 456x456, puis le réseau est affiné en deux phases (tête de classification, puis fine-tuning complet à faible learning rate, en mixed precision pour exploiter le GPU). Le transfert de connaissances depuis ImageNet permet d'atteindre une précision nettement supérieure aux modèles entraînés de zéro.

Indicateur	Valeur
Modèle	EfficientNetB5_TL
Résolution d'entrée	456 x 456
Précision de test	97.91 %
Perte de test	0.071
Paramètres	28 534 017
Époques (fine-tuning)	15 / 15

Table 7. Transfer learning (précision maximale).

Ce modèle atteint 97.91 % de précision — le meilleur résultat du projet.

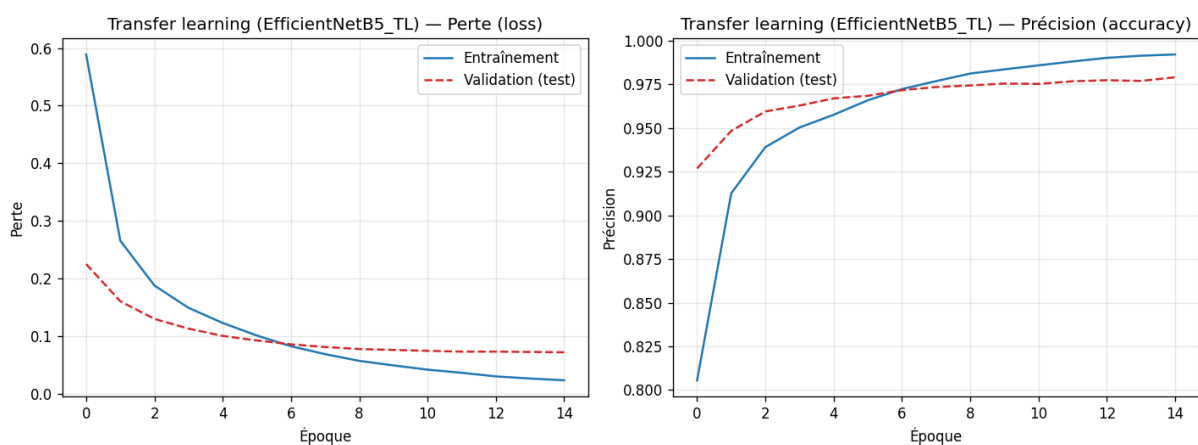


Figure. Courbes du modèle transfer learning.

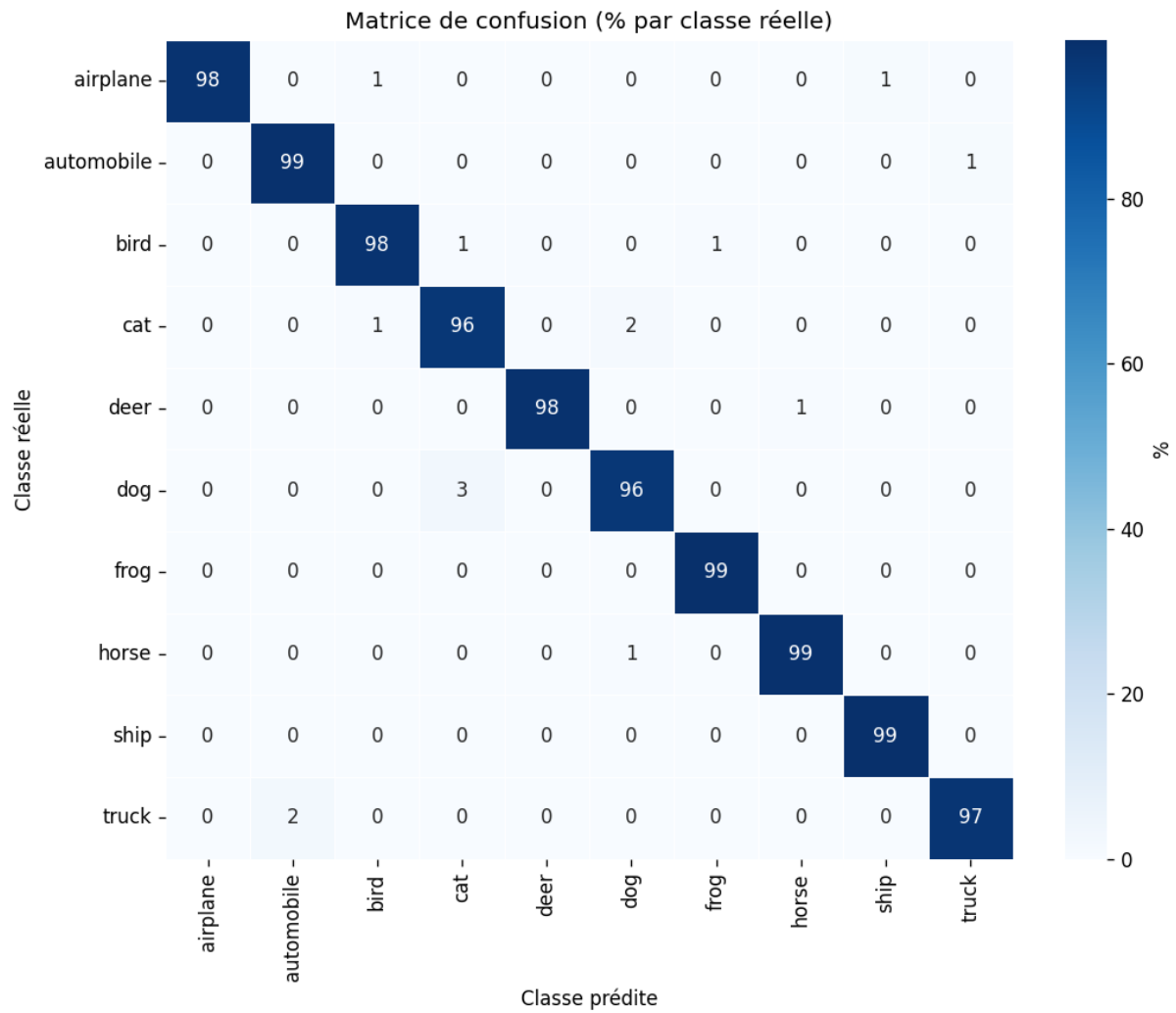


Figure. Matrice de confusion (transfer learning, %).

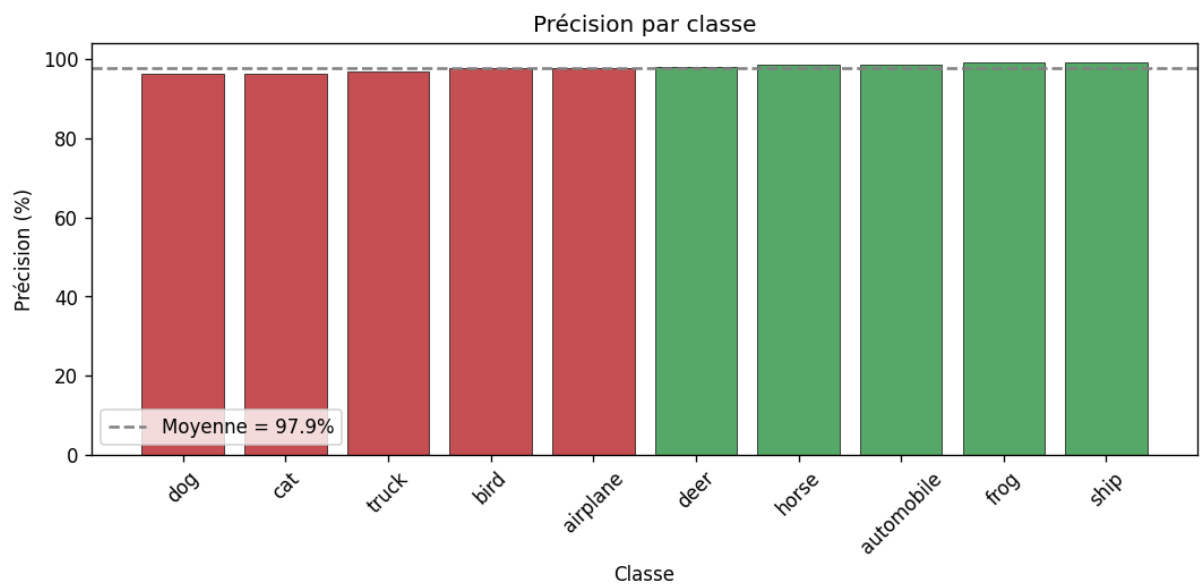


Figure. Précision par classe (transfer learning).

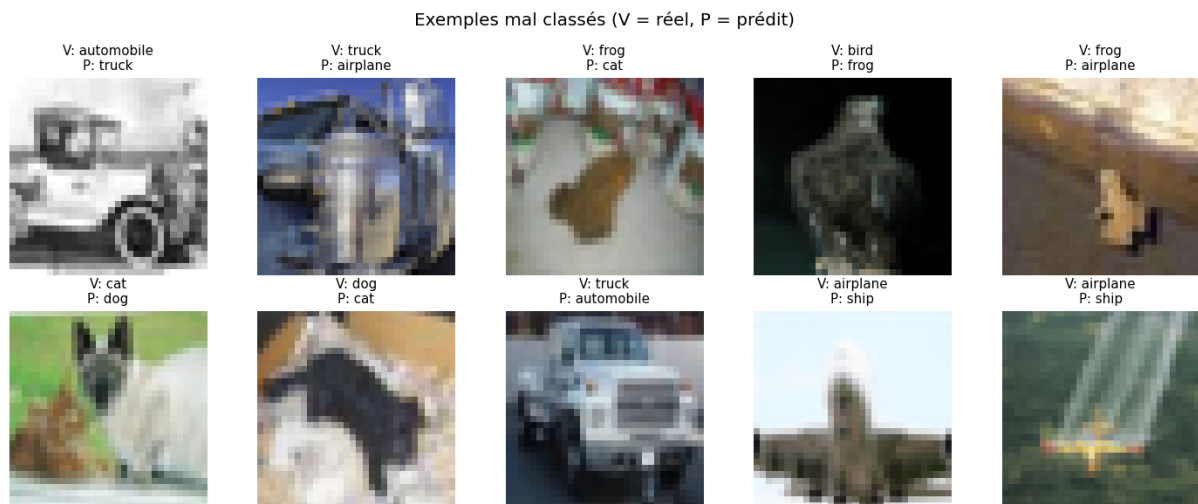


Figure. Exemples mal classés (transfer learning).

8. ML Playground : mise à l'épreuve du modèle

Phase de déploiement de la méthodologie CRISP-DM : le modèle de transfer learning (section 7) est mis en production dans ML Playground, une application web développée pour le projet (backend FastAPI, frontend statique, déploiement Docker), accessible en ligne sur <https://machine-learning.bellahsene.org>. L'interface est adaptée à la fois au web et au mobile.

De l'entraînement à l'usage réel : le modèle entraîné et évalué sur Colab (sections 4 à 7) est sauvegardé au format Keras avec son pré-traitement intégré au graphe (agrandissement 32x32 -> 456x456 et normalisation EfficientNet). L'application n'a donc qu'à lui envoyer une image 32x32 brute : un backend FastAPI charge le modèle et expose une API de prédiction, un frontend léger interroge cette API, et le tout est conteneurisé avec Docker puis déployé sur un serveur auto-hébergé. La théorie -- courbes d'apprentissage, matrices de confusion -- devient ainsi un usage concret : chaque clic des joueurs déclenche une vraie inférence du modèle. Trois mini-jeux mettent le modèle à l'épreuve, chacun sous un angle différent :

- Le Duel -- humain contre machine : 10 manches, 3 secondes par image pour choisir la bonne classe parmi trois propositions ; l'IA répond en parallèle à la même image et les scores sont comparés en direct.
- Dessine, je devine : l'utilisateur dessine la classe imposée sur un canvas et le modèle devine en temps réel, trait après trait. Parfois quelques bonnes couleurs suffisent pour qu'il trouve ; parfois il faut être plus artistique et pointu, car un dessin est très loin des photos 32x32 vues à l'entraînement.
- Test Ultime CINIC-10 : partie pensée pour le correcteur, qui teste le modèle sur des images génériques issues de CINIC-10 (dérivées d'ImageNet) qu'il n'a JAMAIS vues à l'entraînement. C'est l'épreuve la plus difficile : ces images ne suivent pas toujours les mêmes critères de catégorisation que CIFAR-10, et l'enjeu est de repérer quand le modèle se trompe.



Le Duel

Toi contre la machine. 3 secondes par image, 10 manches — qui reconnaît le mieux ?

[Jouer →](#)



Dessine je devine

Dessine le mot impose et regarde le modèle deviner — ou paniquer — trait après trait.

[Jouer →](#)



Test Ultime CINIC-10

Des images qu'il n'a jamais vues. Sauras-tu repérer quand il se trompe ?

[Jouer →](#)



Le Duel

12 joueurs · scrolle pour voir au-delà du top 10

A+	EncoreUnHugo	20359 pts
A	Mehdi	18740 pts
A	mimene	16407 pts
A-	UKM	8046 pts
A-	Mounia la star	7421 pts
B+	triks	900 pts
B+	BAAAW	877 pts
B	Dr Sara	784 pts
B	Dorado	784 pts
B	Nono	784 pts



Dessine je devine — les plus rapides

13 dessinateurs · scrolle pour voir au-delà du top 10

A+	 Mehdi avion	0.77 s
A	 triks oiseau	1.01 s
A	 harry le malicie oiseau	1.32 s
A	 EncoreUnHugo automobile	1.32 s
A-	 Dr Sara avion	1.67 s
B+	 N bateau	1.80 s
B+	 Zid ane oiseau	2.06 s
B	 UKM bateau	2.46 s

Figure. Screen du jeu.



Figure. Screen du jeu.

L'application a très bien fonctionné : étudiants, amis et famille l'ont testée et se sont mis au défi les uns les autres, comme en témoignent les classements de la page d'accueil. C'est aussi ce qui m'a fait le plus plaisir dans ce projet : voir concrètement le résultat de l'effort de la puce A100 -- un modèle entraîné pendant des heures devenu un jeu que chacun peut défier depuis son navigateur ou son téléphone.

9. Conclusion générale

Ce projet a déroulé une démarche complète de classification d'images (CRISP-DM). Le prétraitement (normalisation, one-hot) est indispensable à un entraînement stable. Les VGG dépassent largement CNN1 (LeNet-5), la profondeur améliorant l'extraction de caractéristiques.

Adam est le plus efficace à faible learning rate mais sensible (il diverge à 0.01 et 0.1) ; SGD est plus lent mais robuste. Une valeur faible (0.001) est le meilleur compromis. Dropout et Batch Normalization limitent le surapprentissage ; la BN autorise en plus des learning rates élevés.

Le modèle retenu (VGG3+D+BN / Adam / LR=0.001) atteint 84.81 % sur le test.

En complément (hors énoncé), un modèle optimisé entraîné de zéro (augmentation + filtres croissants) atteint 96.64 %.

Enfin, le transfer learning (EfficientNetB5_TL pré-entraîné sur ImageNet, affiné) porte la précision à 97.91 % — le meilleur résultat du projet, illustrant la puissance du transfert de connaissances par rapport à un entraînement de zéro.

Pistes : architectures résiduelles (ResNet), recherche systématique des hyperparamètres, augmentation avancée (MixUp, RandAugment).

10. Références

- Simonyan, K. & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition (VGG). arXiv:1409.1556.
- Kingma, D. P. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR.
- Srivastava, N. et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR 15.
- Ioffe, S. & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML.
- Tan, M. & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML.
- Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images (CIFAR-10).
- LeCun, Y. et al. (1998). Gradient-Based Learning Applied to Document Recognition (LeNet-5).
- Analytics Vidhya (2021). A Comprehensive Guide on Deep Learning Optimizers.